数学知识
- 质数
  - 试除法判定质数
  - 分解质因数
  - 筛质数
- 快速幂
  - 快速幂
  - 快速幂求逆元
- 求组合数
  - 1
  - 2
  - 3
  - 4

数学知识

质数

试除法判定质数

1. 质数的概念：

   对所有大于1的自然数，定义了质数的概念。

   在大于1的整数中，如果只包含1和本身这两个约数，就被叫做质数，或者素数。

2. 质数的判定：

   试除法（时间复杂度O(n)）：

```
#include <iostream>
#include <algorithm>

using namespace std;

bool is_prime(int n)
{
    if (n < 2) return false;
    for (int i = 2; i < n; i ++)
        if (n % i == 0) return false;

    return true;
}

int main()
{

}
```

   优化（时间复杂度O(sqrt(n)）依据：d能整除n，n/d也能整除n）12 = 3 x 4，3是12的约数，4也是。并且成对出现。因此我们枚举较小的数就行（d < = n/d），整理一下，d就只需要枚举到根号n就可以了

```
#include <iostream>
#include <algorithm>

using namespace std;

bool is_prime(int n)
{
    if (n < 2) return false;
    for (int i = 2; i <= n / i; i ++)
        if (n % i == 0) return false;

    return true;
}

int main()
{

}
```

注意： i < n / i不推荐写成sqrt(n)，因为求根号的运算比较慢；也不推荐写成i * i < n，因为可能存在i * i溢出，导致变成一个负数，影响最终的答案

分解质因数

这个也是用试除法实现的，基本思路：

```
#include <iostream>
#include <algorithm>

using namespace std;

void devide(int n)
{
    for (int i = 2; i <= n / i; i ++)
        if (n % i == 0) // i一定是质数
        {
            int s = 0;
            while(n % i == 0)
            {
                n /= i;
                s ++;
            }

            printf("%d %d\n", i, s);
        }

    if (n > 1) printf("%d %d\n", n, 1);
    puts("");
}

int main()
{
    int n;
```

```
    scanf("%d", &n);
    while(n --)
    {
        int x;
        scanf("%d", &x);
        devide(x);
    }

    return 0;
}
```

注意：if (n % i == 0) //n已经把2到i-1的质因子除完了，因此n不包含前面的所有质因子；又由于n是i的倍数，i肯定也不包含。i一定是质数

n中最多只包含一个大于sqrt(n)的质因子：因为如果有两个，两个乘一块一定大于n了

所以枚举的时候可以先把小于sqrt(n)的质因子先枚举出来（时间复杂度介于log n (因为当n为2 ^ n时，除完后n为1，结束)到根号n)

```
#include <iostream>
#include <algorithm>

using namespace std;

void devide(int n)
{
    for (int i = 2; i <= n / i; i ++)
        if (n % i == 0) // i一定是质数
        {
            int s = 0;
            while(n % i == 0)
            {
                n /= i;
                s ++;
            }

            printf("%d %d\n", i, s);
        }

    if (n > 1) printf("%d %d\n", n, 1);
    puts("");
}

int main()
{
    int n;
    scanf("%d", &n);
    while(n --)
    {
        int x;
        scanf("%d", &x);
        devide(x);
    }

    return 0;
```

```
}
```

筛质数

筛法的朴素做法(时间复杂度nlogn)：



删掉所有数的倍数，剩下的数就是质数：对于剩下的p而言，因为从2到p-1都没有把p删掉，说明p不是2到p - 1之间任何数的倍数

```
#include <iostream>
#include <algorithm>

using namespace std;

const int N = 1000010;

int primes[N], cnt;
bool st[N];

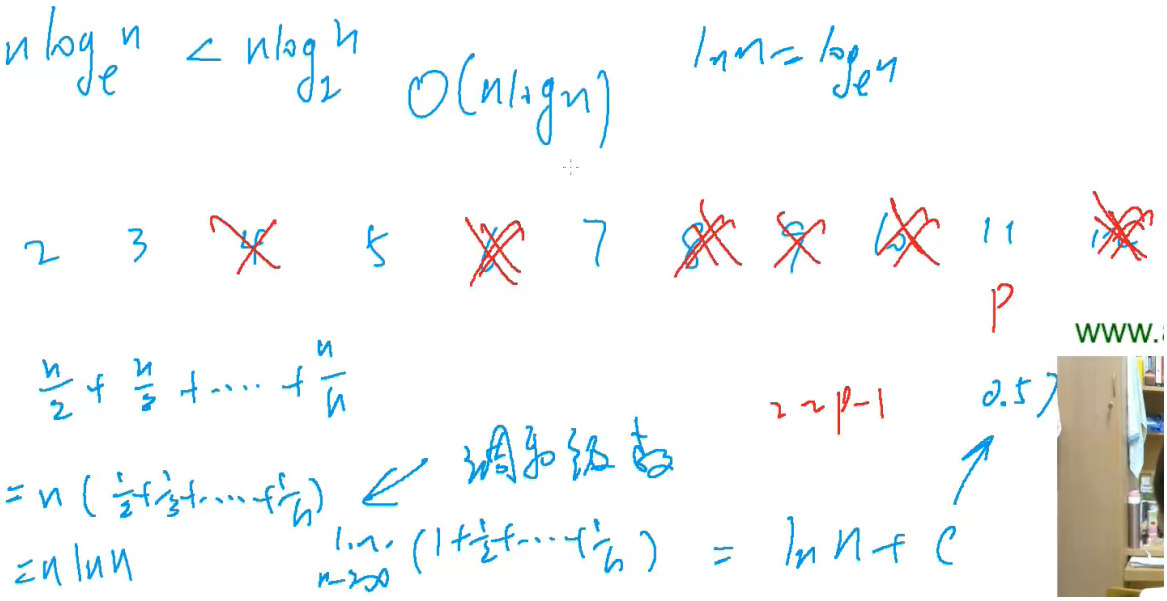
void get_primes(int n)
{
    for (int i = 2; i <= n; i ++)
    {
        if (!st[i])
        {
            primes[cnt ++] = i;
        }

        for (int j = i + i; j <= n; j += i) st[j] = true;
    }
}

int main()
{
    int n;
    cin >> n;
    get_primes(n);

    cout << cnt << endl;

    return 0;
}
```



改进埃氏筛法：把所有质数的倍数删掉就可以了(因为数包含：质数、合数，而合数是质数的倍数)

```
#include <iostream>
#include <algorithm>

using namespace std;

const int N = 1000010;

int primes[N], cnt;
bool st[N];

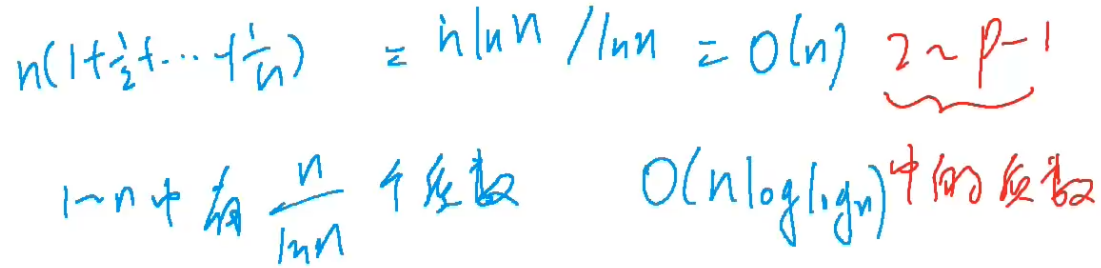
void get_primes(int n)
{
    for (int i = 2; i <= n; i ++)
    {
        if (!st[i])
        {
            primes[cnt ++] = i;
            for (int j = i + i; j <= n; j += i) st[j] = true;
        }


    }
}

int main()
{
    int n;
    cin >> n;
    get_primes(n);

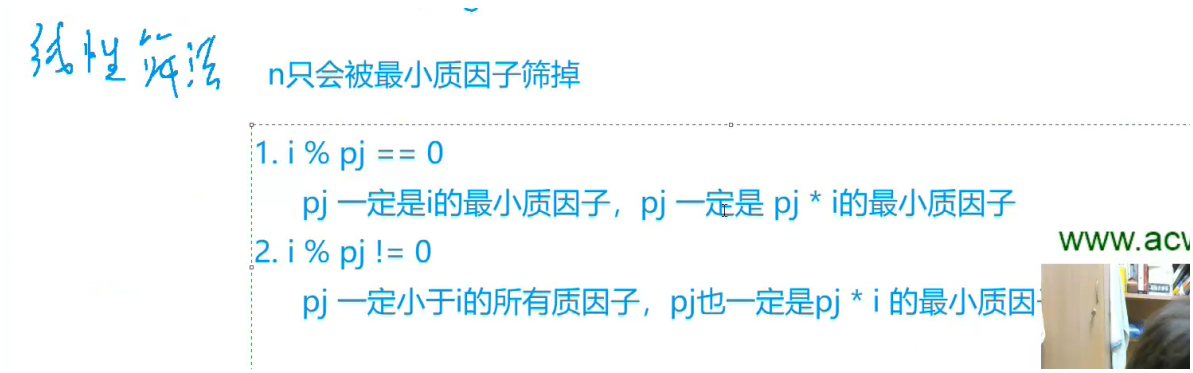
    cout << cnt << endl;

    return 0;
}
```



线性筛法：核心是每个合数只能被它的最小质因子筛掉

```
void get_primes(int n)
{
    for (int i = 2; i <= n; i ++ )
    {
        if (!st[i]) primes[cnt ++ ] = i;
        for (int j = 0; primes[j] <= n / i; j ++ )
        {
            st[primes[j] * i] = true;
            if (i % primes[j] == 0) break;      // primes[j]一定是i的最小质因子
        }
    }
}
```




```
#include <iostream>
#include <algorithm>

using namespace std;

const int N = 1000010;

int primes[N], cnt;
bool st[N];

void get_primes(int n)
{
    for (int i = 2; i <= n; i ++)
    {
        if (!st[i]) primes[cnt ++] = i;
        for (int j = 0; primes[j] <= n / i; j ++)
        {
            st[primes[j] * i] = true;
            if (i % primes[j] == 0) break;
        }
    }
}
```

```
int main()
{
    int n;
    cin >> n;
    get_primes(n);

    cout << cnt << endl;

    return 0;
}
```

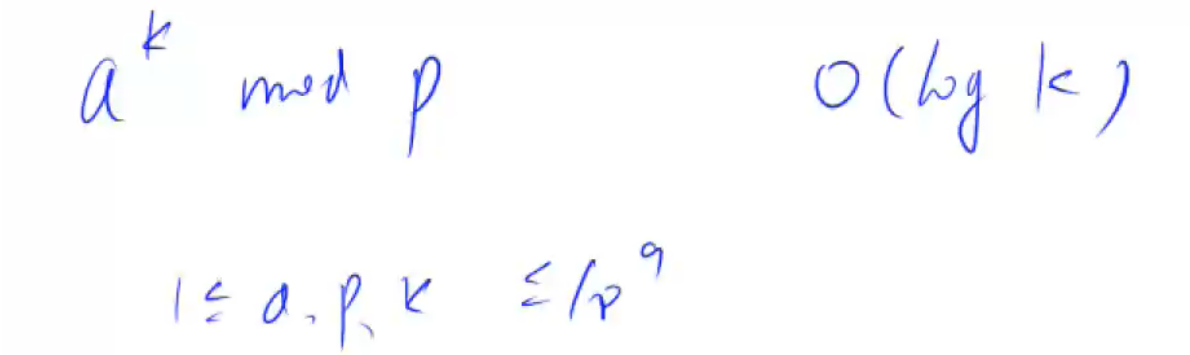
注意：这里解释一下为什么for (int j = 0; primes[j] <= n / i; j ++)不用加上j < cnt的判断，可以将i拆分为质数和合数两种情况看，当前primes[j] <= n / i的判断都不会导致越界

damn，看来还是要写个详细解释：

当i是合数，在枚举primes[j]的时候一定会被它的最小质因子筛下来；当i是质数，当primes[j] = i时，自己就是自己的最小质因子，也会停下来。

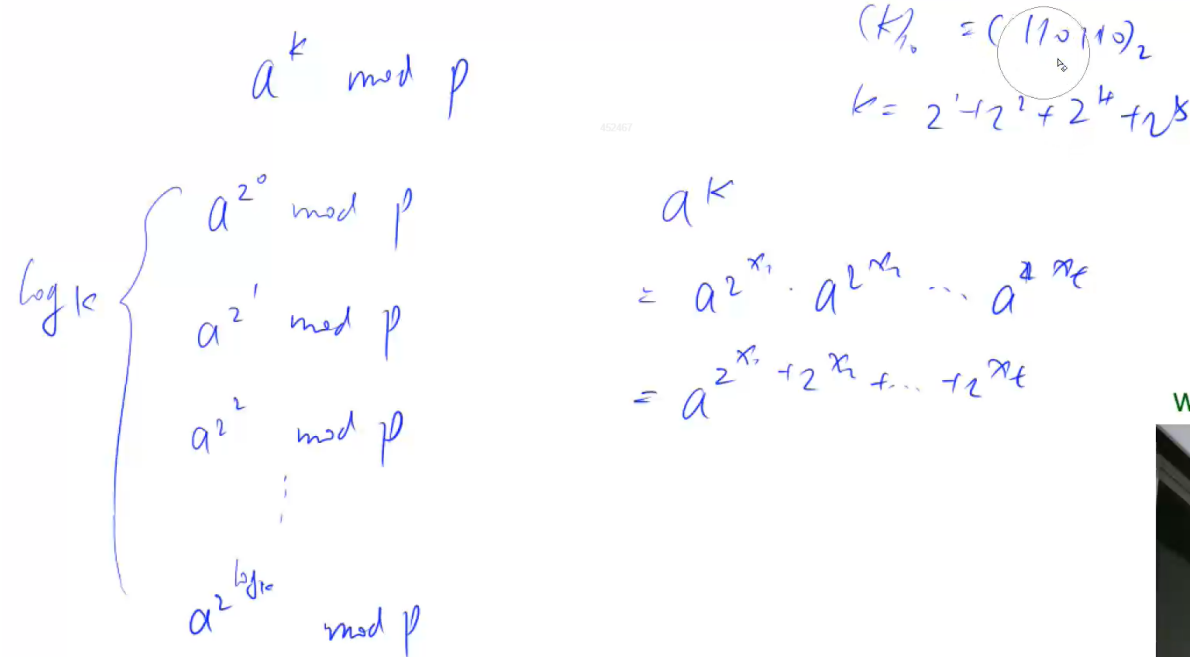
快速幂

快速幂



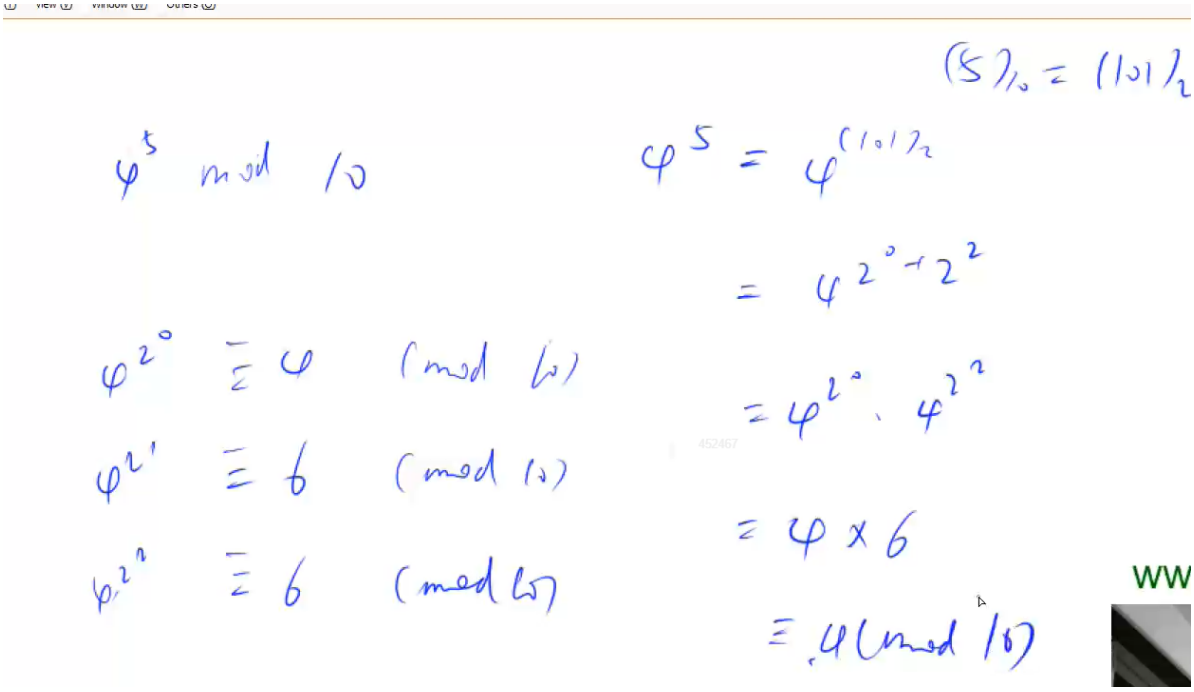
快速幂的核心思路是：

数据预处理



把k表示成二进制，然后加上就行

举个例子：




```cpp
#include <iostream>
#include <algorithm>

using namespace std;

typedef long long LL; // 有乘积操作，那两个10^9相乘会爆int

// a^k % p
int qmi(int a, int k, int p)
{
    int res = 1;
    while(k)
    {
        if (k & 1) res = (LL)res * a % p; //检查k的末位是不是1
```

```
        k >>= 1; //将右移后的结果再赋值给 `k`
        a = (LL)a * a % p;
    }

    return res;
}

int main()
{
    int n;
    scanf("%d", &n);
    while(n --)
    {
        int a, k, p;
        scanf("%d%d%d", &a, &k, &p);

        printf("%d\n", qmi(a, k, p));
    }

    return 0;
}
```

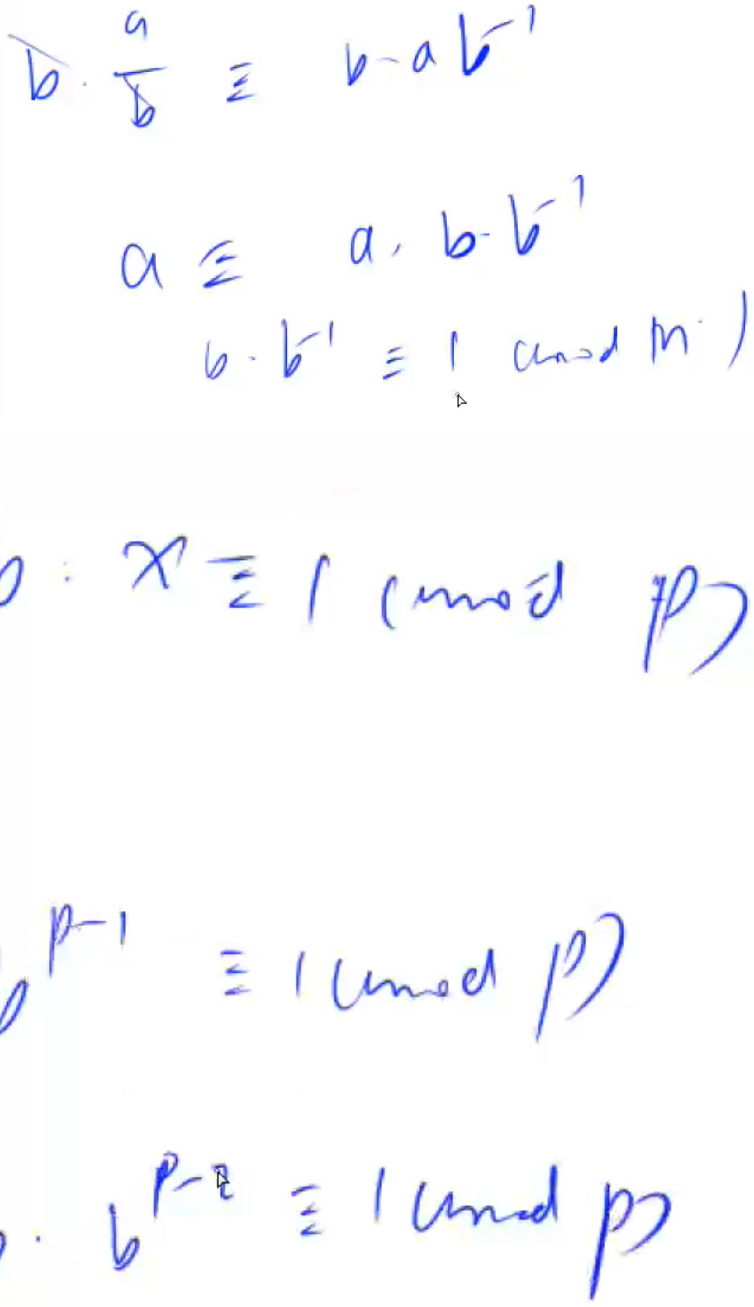
`k >> 1` 表示将 `k` 的二进制表示向右移动1位，相当于将 `k` 除以2，并忽略小数部分（即向下取整）。

`k >>= 1` 是将右移后的结果再赋值给 `k`，相当于 `k = k >> 1`。

快速幂求逆元

$$\frac{a}{b} \equiv a \cdot x \pmod{m}$$

$$\frac{a}{b} \equiv a \cdot b^{-1}$$



费马定理↑ 如果b和p互质，b^(p - 1) mod p = 1

```
#include <iostream>
#include <algorithm>

using namespace std;

typedef long long LL; // 有乘积操作，那两个10^9相乘会爆int

// a^k % p
int qmi(int a, int k, int p)
{
    int res = 1;
    while(k)
    {
        if (k & 1) res = (LL)res * a % p;
        k >>= 1;
```

```
        a = (LL)a * a % p;
    }

    return res;
}

int main()
{
    int n;
    scanf("%d", &n);
    while(n --)
    {
        int a, p;
        scanf("%d%d", &a, &p);


        if (a % p)
        {
            int res = qmi(a, p - 2, p);
            printf("%d\n", res);
        }
        else puts("impossible");
    }

    return 0;
}
```

注意：b是p的倍数时无解

求组合数

1

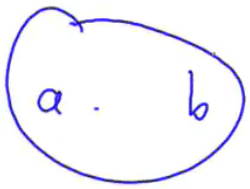
组合数有很多种题，要根据数据范围来选有哪些方式

这个课的初衷是像背单词一样学算法，除了理解之外还要课下不断去记忆，默写出来我们的算法

$$C_a^b = \frac{a \times (a-1) \times \cdots \times (a-b+1)}{1 \times 2 \times 3 \times \cdots \times b} = \frac{a!}{b!\,(a-b)!}$$

$$C_6^2 = \frac{6 \times 5}{2 \times 1} = \frac{6!}{2! \times (6-2)!} = \frac{6!}{2! \times 4!} = 15$$

$$C_a^b = C_{a-1}^b + C_{a-1}^{b-1}$$




```
#include <iostream>
#include <algorithm>

using namespace std;

const int N = 2010, mod = 1e9 + 7;

int c[N][N];

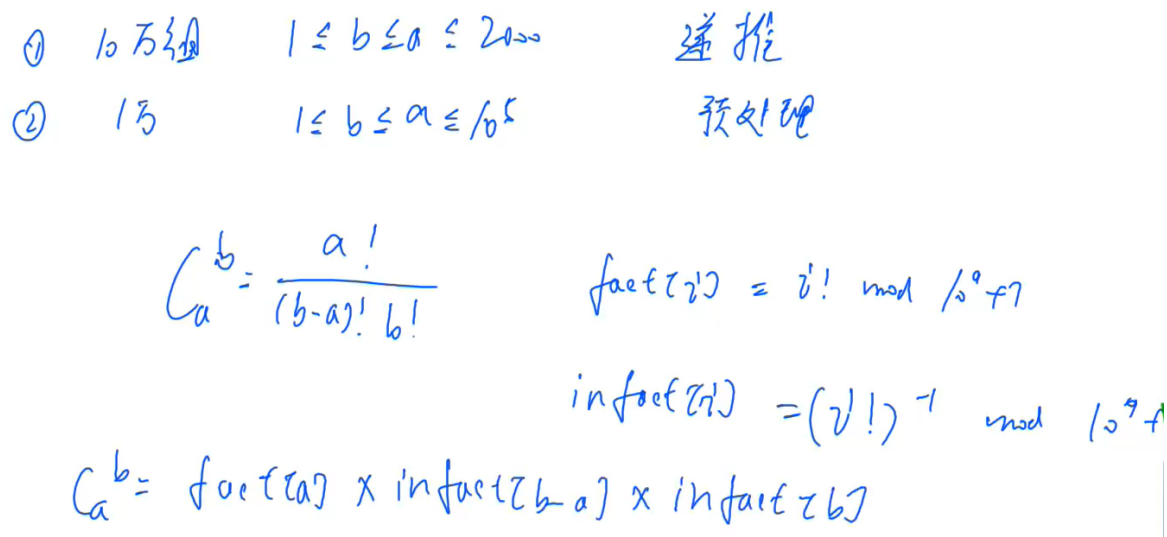
void init()
{
    for (int i = 0; i < N; i ++)
        for (int j = 0; j <= i; j ++)
            if (!j) c[i][j] = 1;
            else c[i][j] = (c[i - 1][j] + c[i - 1][j - 1]) % mod;
}

int main()
{
    init();

    int n;
    scanf("%d", & n);
    while(n --)
    {
        int a, b;
        scanf("%d%d", &a, &b);
        printf("%d\n", c[a][b]);
    }
```

```
    return 0;
}
```

2




```
#include <iostream>
#include <algorithm>

using namespace std;

typedef long long LL;

const int N = 100010, mod = 1e9 + 7;

int fact[N], infact[N];

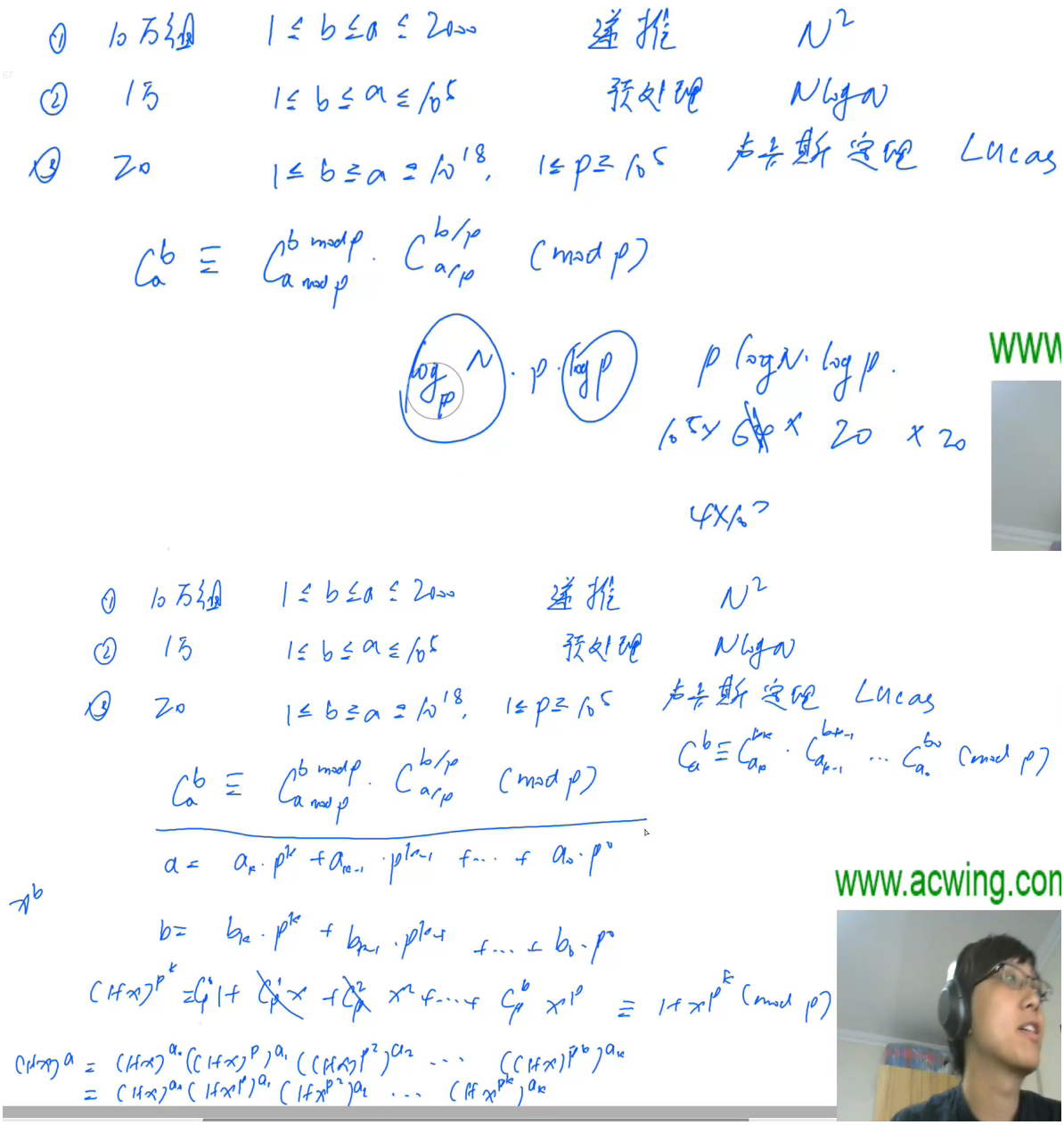
int qmi(int a, int k, int p)
{
    int res = 1;
    while(k)
    {
        if (k & 1) res = (LL)res * a % p;
        a = (LL) a * a % p;
        k >>= 1;
    }
    return res;
}

int main()
{
    infact[0] = fact[0] = 1;
    for (int i = 1; i < N; i ++)
    {
        fact[i] = (LL)fact[i - 1] * i % mod;
        infact[i] = (LL)infact[i - 1] * qmi(i, mod - 2, mod) % mod;
    }
```

```
    int n;
    scanf("%d", &n);
    while(n --)
    {
        int a, b;
        scanf("%d%d", &a, &b);
        printf("%d\n", (LL)fact[a] * infact[b] % mod * infact[a - b] % mod);
    }

    return 0;
}
```

3




```
#include <iostream>
#include <algorithm>

using namespace std;
```

```
typedef long long LL;
int p;

int qmi(int a, int k, int p)
{
    int res = 1;
    while(k)
    {
        if (k & 1) res = (LL)res * a % p;
        a = (LL)a * a % p;
        k >>= 1;
    }

    return res;
}

int C(LL a, LL b)
{
    int res = 1;
    for (int i = 1, j = a; i <= b; i ++, j --)
    {
        res = (LL)res * j % p;
        res = (LL)res * qmi(i, p - 2, p) % p;
    }

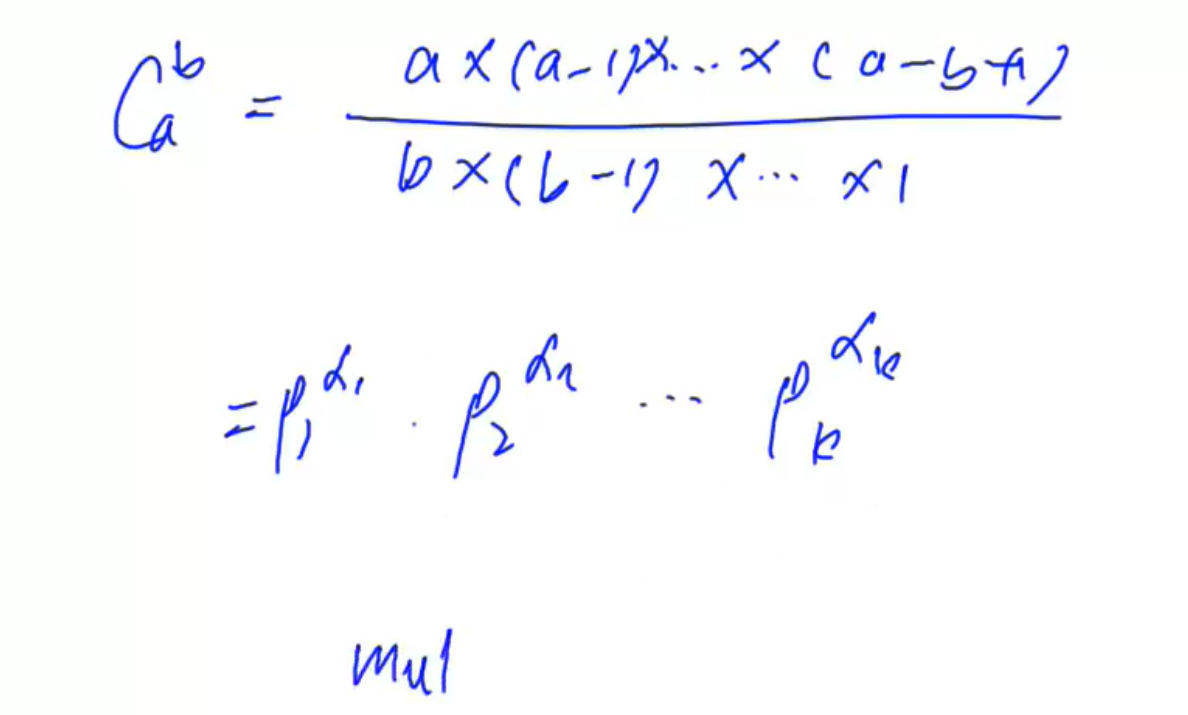
    return res;
}

int lucas(LL a, LL b)
{
    if (a < p && b < p) return C(a, b);
    return (LL)C(a % p, b % p) * lucas(a / p, b / p) % p;
}

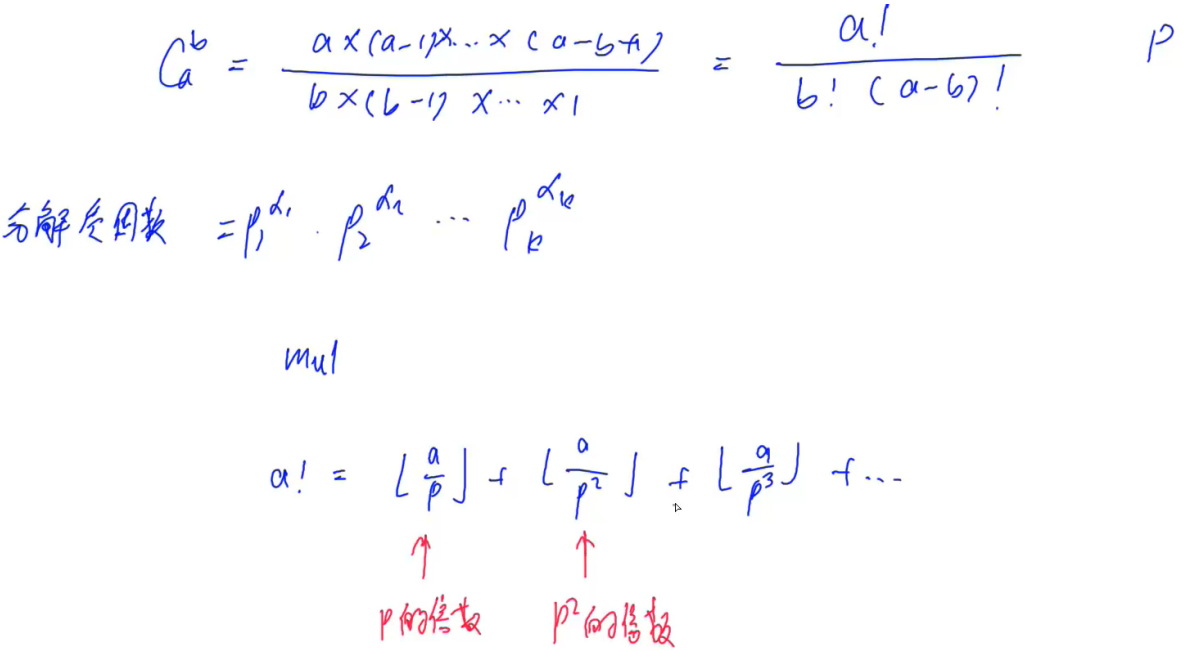
int main()
{
    int n;
    cin >> n;
    while(n --)
    {
        LL a, b;
        cin >> a >> b >> p;
        cout << lucas(a, b) << endl;
    }

    return 0;
}
```

4



分解质因数就变成乘法了



算法思路：

1.筛质数

2.求质数的次数

3.高精度乘法

```
#include <iostream>
#include <algorithm>
#include <vector>

using namespace std;

const int N = 5010;
```

```
int primes[N], cnt;
int sum[N];
bool st[N];

void get_primes(int n)
{
    for (int i = 2; i <= n; i ++)
    {
        if (!st[i]) primes[cnt ++] = i;
        for (int j = 0; primes[j] <= n / i; j ++)
        {
            st[primes[j] * i] = true;
            if (i % primes[j] == 0) break;
        }
    }
}

int get(int n, int p)
{
    int res = 0;
    while(n)
    {
        res += n / p;
        n /= p;
    }
    return res;
}

vector<int> mul(vector<int> a, int b)
{
    vector<int> c;
    int t = 0;
    for (int i = 0; i < a.size(); i ++)
    {
        t += a[i] * b;
        c.push_back(t % 10);
        t /= 10;
    }

    while(t)
    {
        c.push_back(t % 10);
        t /= 10;
    }

    return c;
}

int main()
{
    int a, b;
    cin >> a >> b;

    get_primes(a);
```

```
    for (int i = 0; i < cnt; i ++)
    {
        int p = primes[i];
        sum[i] = get(a, p) - get(b, p) - get(a - b, p);
    }

    // 下面用一个高精度的乘法把所有质因子乘起来就行
    vector<int> res;
    res.push_back(1);

    for (int i = 0; i < cnt; i ++)
        for (int j = 0; j < sum[i]; j ++)
            res = mul(res, primes[i]);

    for (int i = res.size() - 1; i >= 0; i --) printf("%d", res[i]);
    puts("");

    return 0;
}
```